Omilia
Conversational Intelligence

Programming Assignment

Natural Numbers Interpretation

Background

When interacting with an automated dialogue system, the caller is often required to provide their phone number. Phone numbers may be spoken in many ways with different digit groupings (e.g. 2106930664 may be spoken as "210 69 30 6 6 4" or "210 69 664"). Furthermore, speech input may cause ambiguities. For example, if the caller says "twenty-five", this could be transcribed as "25" or "20 5".

The application you are asked to develop deals with some of these issues.

Task

Implement an application that handles natural numbers interpretation. **You may use any programming language you are comfortable with** (e.g. Python, Java, C#, JavaScript, etc.).

The application should ask the user for a number and then present the identified number back to the user. It should accept as input a sequence of numbers separated by a space character. Each number in the sequence may have up to three digits. This input represents recognized numbers that a user may pronounce, for example:

- 30 2 5 58
- 2 10 69 30 6 60 4
- 6 97 400 23 7 40 5

The application should then identify and print the input number:

Input	Output
30 2 5 58	302558
2 10 69 30 6 60 4	21069306604

It should be a standalone application, running either from the command line or with a simple GUI.

Base Level: Phone Number Validation

The application should state whether the output number is a valid Greek telephone number. Assume that valid Greek phone numbers have either 10 or 14 digits:

- **10 digits:** must start with '2' or '69'
- **14 digits:** must start with '00302' or '003069'

Examples:

Input	Output	Phone number
30 2 5 58	302558	INVALID
2 10 69 30 6 6 4	2106930664	**VALID**
2 10 69 30 6 60 4	21069306604	INVALID
0 0 30 69 74 0 9 22 52	00306974092252	**VALID**

Advanced Level: Identify Natural Number Ambiguities

The application should check for possible ambiguities in number spelling. For example, if the input sequence contains '... 20 5 ...', the resulting number may be either '...205...' or '...25...'. Likewise, if the sequence contains '...75...', the result may be '...705...' or '...75...'.

The application should identify these ambiguities and return all possible number interpretations.

Example 1 — input: 2 10 6 9 30 6 6 4

Interpretation	Output	Phone number
1	2106930664	**VALID**
2	210693664	INVALID

Example 2 — input: 2 10 69 30 6 6 4

Interpretation	Output	Phone number
1	2106930664	**VALID**
2	210693664	INVALID
3	2106093664	**VALID**
4	21060930664	INVALID

Example 3 — input: 0 0 30 69 700 24 1 3 50 2

Interpretation	Output	Phone number
1	0030697002413502	INVALID
2	003069700241352	INVALID
...	...	...
n	00306972413502	**VALID**
...	...	...
m	00306097241352	INVALID

Submission

Please share your source code (e.g. a Git repository link or a zip file), together with a short README stating the language/version used and how to build and run the application.